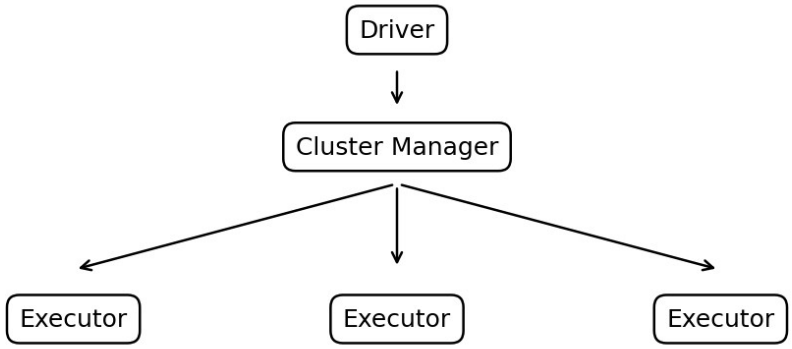
35. SPARK

Distributed Computing • PySpark • DataFrame API • Shuffle • Joins • Optimization

Spark yüksek seviye mimarisi



Driver planlar; executor'lar task'leri partition'lar üzerinde çalıştırır.

Bu modülün amacı
Tek makine sınırına yaklaşan veriyi Spark ile nasıl dağıtık işlediğini anlamak. Amaç sadece PySpark syntax ezberlemek değil; partition, shuffle, execution plan, join ve storage kararlarının neden performansı belirlediğini kavramaktır.

Kazanım	Modül sonunda
Mimari	Driver, executor, cluster manager ve task ilişkisini açıklayabilirsin.
DataFrame API	PySpark ile select/filter/groupBy/join/window akışları kurabilirsin.
Lazy execution	Transformation/action ve DAG mantığını anlayabilirsin.
Partition/shuffle	Dağıtık performans darboğazlarını teşhis edebilirsin.
Optimization	Predicate pushdown, column pruning, broadcast join ve cache'i doğru kullanabilirsin.
ML/data science	Spark'ı büyük ölçekli feature engineering ve ETL içinde konumlandırabilirsin.

1. Spark nedir?

Apache Spark, büyük veri üzerinde distributed computation için tasarlanmış bir compute engine'dir. Veriyi birçok worker üzerinde partition'lara bölerek paralel işler.

Problem	Spark yaklaşımı
Tek makine RAM/CPU yetmiyor	Veriyi cluster'a dağıt
Milyarlarca satır transform	Partition başına paralel task
Tekrarlı ETL	Distributed execution plan
SQL + Python ihtiyacı	DataFrame/SQL API

Spark bir database değildir
Spark compute engine'dir. Veriyi object storage, warehouse, HDFS benzeri storage katmanlarından okuyup işler.

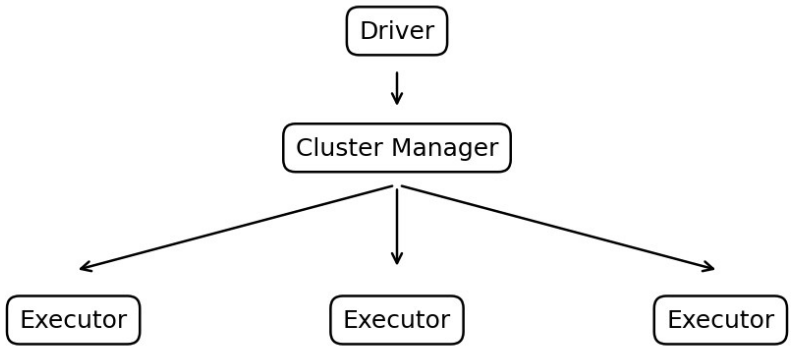
2. Ne zaman Spark kullanılır?

- Veri tek makinede rahat işlenemiyorsa.
- Çok büyük join/groupBy/aggregation varsa.
- Distributed ETL/feature engineering gerekiyorsa.
- Cluster zaten şirket altyapısının parçasıysa.

Ne zaman gereksiz?
5 GB veri Pandas/Polars ile saniyeler-dakikalar içinde çözülüyorsa Spark cluster overhead'i gereksiz olabilir.

3. Mimari: Driver ve Executor

Spark yüksek seviye mimarisi



Driver planlar; executor'lar task'leri partition'lar üzerinde çalıştırır.

Bileşen	Görev
Driver	Programı çalıştırır, plan oluşturur, task'leri koordine eder

Cluster Manager	Kaynak tahsis eder
Executor	Task çalıştırır, partition verisini işler
Task	Bir stage içindeki en küçük execution unit

4. SparkSession

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("sales-etl")
    .getOrCreate()
)
```

Giriş noktası
Modern PySpark uygulamalarında DataFrame/SQL işlemlerinin ana entry point'i SparkSession'dır.

5. DataFrame nedir?

Schema'lı, dağıtık bir tablo abstraction'ıdır. Kullanıcı logical query yazar; Spark optimizer bunu physical plan'a dönüştürür.

```
df = spark.read.parquet(
    "s3://bucket/orders/"
)

df.printSchema()
df.show(5)
```

6. Schema

Tip	Örnek
StringType	customer_id
IntegerType	quantity
DoubleType	amount
TimestampType	event_time
Array/Struct	Nested JSON data

Explicit schema
CSV/JSON gibi kaynaklarda schema inference yerine production pipeline'da explicit schema daha güvenilir ve hızlıdır.

7. select ve withColumn

```
from pyspark.sql import functions as F

out = (
    df.select(
        "customer_id",
        "amount",
        "order_ts"
    )
    .withColumn(
        "order_date",
        F.to_date("order_ts")
    )
)
```

8. filter / where

```
valid = df.filter(
    (F.col("amount") > 0)
    & F.col("customer_id").isNotNull()
)
```

Pushdown
Parquet gibi columnar kaynaklarda bazı filter'lar storage scan'e kadar push down edilerek gereksiz veri okuma azaltılabilir.

9. groupBy ve aggregate

```
daily = (
    df.groupBy(
        "store_id",
        F.to_date("order_ts").alias("date")
    )
    .agg(
        F.sum("amount").alias("revenue"),
        F.countDistinct("order_id").alias("orders")
    )
)
```

Wide operation
groupBy aynı key'e ait kayıtları aynı partition'a taşımak zorunda olduğu için shuffle yaratabilir.

10. Window functions

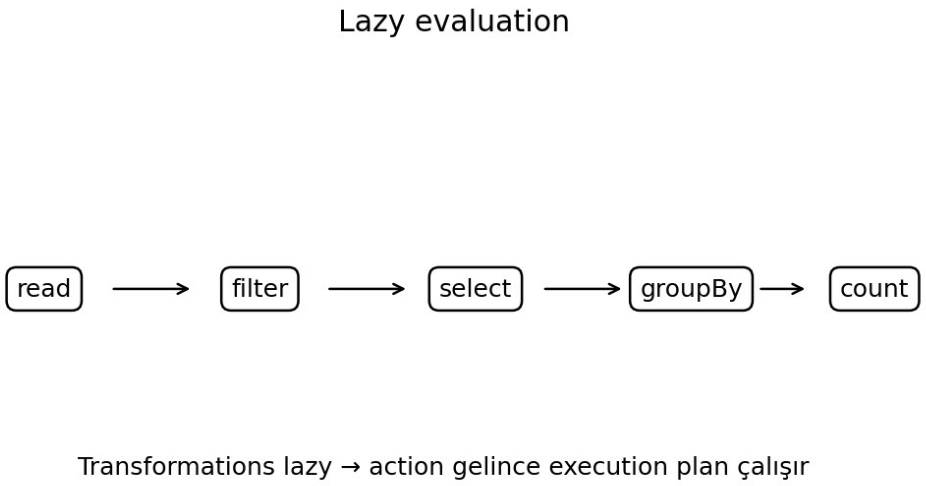
```
from pyspark.sql.window import Window

w = (
    Window
    .partitionBy("customer_id")
    .orderBy("order_ts")
)

df2 = df.withColumn(
    "prev_amount",
    F.lag("amount", 1).over(w)
)
```

Window ihtiyacı	Örnek
Lag/lead	Previous transaction
Rank	Top-N products
Rolling	Recent behavior
Cumulative	Lifetime spend

11. Transformation vs Action



Tür	Örnek
Transformation	select, filter, withColumn, join, groupBy
Action	count, collect, write, show

Lazy evaluation
Transformation çağrıları hemen compute etmez. Action gelene kadar logical plan birikir; Spark optimize ettikten sonra çalıştırır.

12. collect() tehlikesi

collect tüm satırları driver memory'sine çeker.

```
# büyük DataFrame'de tehlikeli
rows = df.collect()
```

Kural
Distributed veriyi driver'a taşımak Spark'ın ölçek avantajını yok eder. Küçük sonuç dışında collect kullanma.

13. explain()

```
df.explain("formatted")
```

Senior alışkanlık
Yavaş pipeline'da önce execution plan'a bak: scan, filter, exchange, sort, join ve stage sınırlarını gör.

14. Catalyst Optimizer

Spark SQL/DataFrame logical plan'ını optimize eden query optimizer katmanıdır.

- Predicate pushdown
- Column pruning
- Constant folding
- Join planning

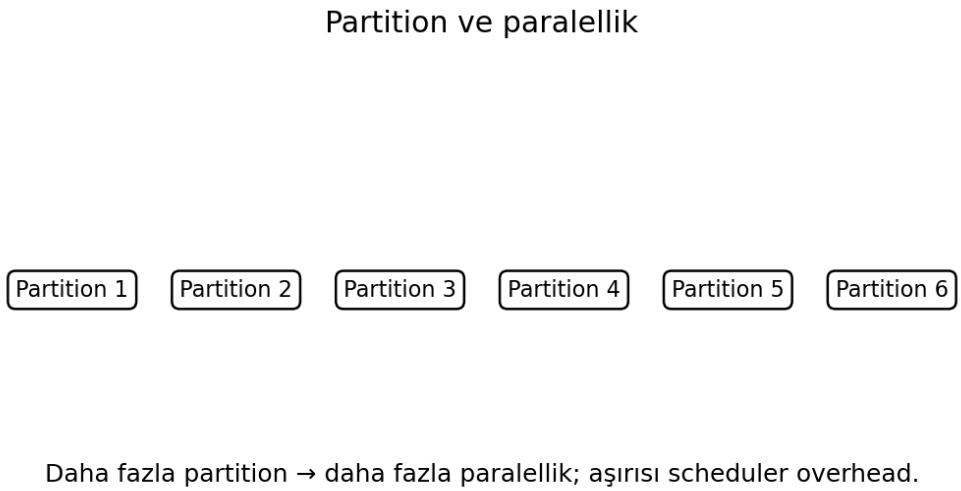
Neden DataFrame?
Optimizer DataFrame/SQL expression'larını anlayabilir; keyfi Python kodunu aynı düzeyde optimize edemez.

15. Tungsten / whole-stage codegen sezgisi

Spark execution layer memory layout, JVM bytecode generation ve CPU verimliliği için optimization uygular.

Pratik sonuç
Python UDF yerine built-in Spark functions kullanmak genellikle daha hızlıdır.

16. Partition



DataFrame fiziksel olarak partition'lara ayrılır. Bir task tipik olarak bir partition üzerinde çalışır.

```
print(df.rdd.getNumPartitions())
```

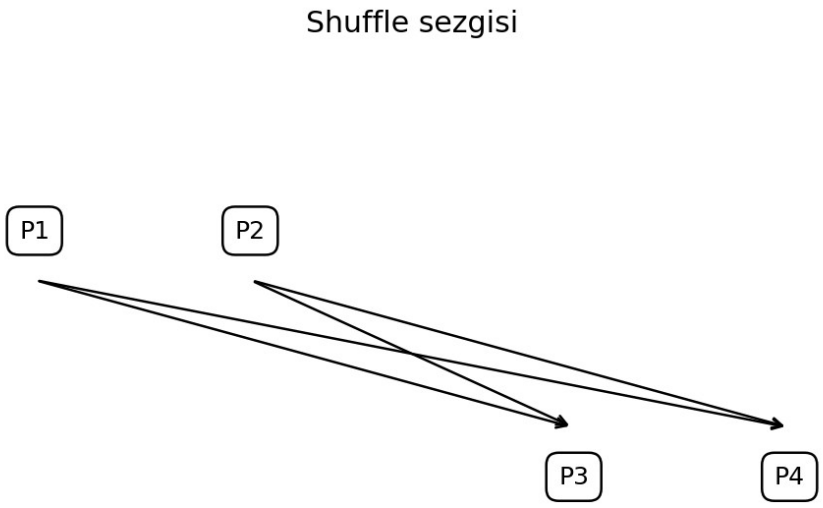
Paralellik
Partition sayısı parallel task sayısını etkiler. Çok az partition düşük paralellik; çok fazla küçük partition overhead yaratır.

17. repartition vs coalesce

Metot	Davranış
repartition(n)	Partition sayısını artırabilir/azaltabilir; genellikle shuffle
coalesce(n)	Genellikle partition azaltır; shuffle'ı minimize etmeye çalışır

```
df = df.repartition(200, "customer_id")
small = df.coalesce(20)
```

18. Shuffle



Shuffle, verinin executor/partition'lar arasında network üzerinden yeniden dağıtılmasıdır.

Shuffle tetikleyebilir	Neden
groupBy	Aynı key'ler birleşmeli
join	Join key'leri colocate edilmeli
distinct	Duplicate karşılaştırılmalı
orderBy	Global ordering gerekir
repartition	Yeni partition distribution

Spark performansının kalbi
Çoğu ciddi performans problemi shuffle volume, skew veya spill ile ilişkilidir.

19. Narrow vs Wide transformations

Tür	Örnek	Özellik
Narrow	filter, select, map	Child partition tek parent'a bağlı
Wide	groupBy, join, distinct	Birden çok parent partition'dan veri gerekir

Stage sınırı
Wide dependency çoğu zaman yeni stage ve shuffle boundary yaratır.

20. Stage ve Task

Spark job, shuffle boundary'lerine göre stage'lere; stage de partition başına task'lere ayrılır.

Seviye	Anlam
Job	Action ile tetiklenen genel çalışma
Stage	Shuffle ile ayrılan task grubu
Task	Bir partition üzerindeki execution

21. Join türleri

Join	Kullanım
inner	Eşleşenler
left	Sol tabloyu koru
full	İki tarafı koru
left_semi	Sağda eşleşme varsa sol satır
left_anti	Sağda eşleşme yoksa sol satır

```
joined = orders.join(
    customers,
    on="customer_id",
    how="left"
)
```

22. Broadcast join

Bir taraf çok küçükse küçük tablo tüm executor'lara kopyalanır; büyük tablo shuffle edilmeden join yapılabilir.

```
from pyspark.sql.functions import broadcast

result = fact.join(
    broadcast(dim_product),
    "product_id"
)
```

Ne zaman?
Dimension tablo küçük, fact tablo çok büyükse güçlü optimizasyondur. Küçük sanılan tablo executor memory'sini aşarsa tersine sorun yaratır.

23. Sort-merge join

İki büyük dataset join olduğunda Spark çoğu durumda key'lere göre shuffle+sort edip merge join kullanabilir.

Maliyet
Network shuffle + sort pahalıdır. Gereksiz geniş kolonları join öncesi düşürmek faydalıdır.

24. Join skew

Bazı key'ler verinin çok büyük kısmını taşıyorsa bir-iki task devleşir; diğer executor'lar boş bekler.

- Hot key
- Null key
- Unknown category
- Popular customer/product

Belirti
Task süreleri çok dengesiz; tek task diğerlerinden kat kat uzun.

25. Skew çözüm yaklaşımları

- Hot key'i ayrı işlemek.
- Salt key tekniği.
- Broadcast join.
- Adaptive Query Execution skew handling.
- Null/unknown key'i ayrı filtrelemek.

Önce ölç
Skew optimizasyonu veri dağılımını görmeden yapılmamalıdır.

26. Adaptive Query Execution (AQE)

Runtime istatistiklerine göre bazı physical plan kararlarını execution sırasında optimize eder.

- Coalesce shuffle partitions
- Skew join optimization
- Join strategy değişimi

Modern Spark
AQE birçok workload'da varsayılan/önemli bir performans katmanıdır.

27. Cache / persist

```
df_cached = df.cache()
df_cached.count()
```

Aynı expensive DataFrame birden fazla action'da tekrar kullanılıyorsa cache faydalı olabilir.

Cache her şeyi hızlandırmaz
Bir kez kullanılan veya çok büyük dataset'i cache etmek memory baskısı ve eviction yaratabilir.

28. Persist seviyeleri

Memory/disk kullanımı için farklı storage level seçenekleri vardır.

- MEMORY_ONLY
- MEMORY_AND_DISK
- DISK_ONLY

Pratik
Dataset memory'ye sığmıyorsa disk fallback daha güvenli olabilir.

29. UDF neden yavaş olabilir?

Python UDF, JVM-Spark execution ile Python process arasında serialization overhead oluşturabilir ve Catalyst optimizasyonunu kısıtlar.

```
# tercihen built-in function
df = df.withColumn(
    "log_amount",
    F.log1p("amount")
)
```

Öncelik sırası
Built-in functions > SQL expressions > pandas UDF > Python UDF, çoğu performans senaryosunda iyi bir genel kuraldır.

30. Pandas UDF

Apache Arrow ile vectorized batch transfer kullanarak klasik Python UDF'den daha verimli olabilir.

Yine de
Built-in Spark function varsa onu kullanmak genellikle daha kolay ve optimize edilebilirdir.

31. Reading Parquet

```
df = (
    spark.read
    .parquet("s3://lake/orders/")
)
```

- Column pruning
- Predicate pushdown
- Partition pruning
- Compression

Spark + Parquet
Dağıtık analytics için en yaygın ve güçlü kombinasyonlardan biridir.

32. Partition pruning

```
daily = (
  spark.read
  .parquet("s3://lake/orders/")
  .filter(F.col("order_date") == "2026-10-10")
)
```

Fiziksel partition
order_date storage partition ise Spark gereksiz tarih dosyalarını açmadan scan'i daraltabilir.

33. File size ve small files

Çok sayıda küçük dosya task scheduling ve file listing overhead yaratır.

- Compaction
- Repartition/coalesce before write
- Reasonable target file size

Çok büyük dosya da sorun
Parallelism düşebilir. Dosya boyutu workload ve storage'a göre dengelenir.

34. Write modes

Mode	Davranış
append	Yeni data ekle
overwrite	Hedefi değiştir
error	Var ise fail
ignore	Var ise atla

```
(
  daily.write
  .mode("overwrite")
  .partitionBy("date")
  .parquet("s3://gold/daily_sales/")
)
```

35. Dynamic partition overwrite sezgisi

Backfill/reprocessing sırasında tüm tablo yerine sadece etkilenen partition'ları overwrite etmek daha güvenli ve ucuz olabilir.

Data engineering bağlantısı
34. modüldeki idempotency ve backfill kavramları Spark write stratejilerinde doğrudan karşına çıkar.

36. SQL kullanmak

```
df.createOrReplaceTempView("orders")

spark.sql(
    '''
    SELECT customer_id,
           SUM(amount) AS revenue
    FROM orders
    GROUP BY customer_id
    '''
)
```

DataFrame vs SQL
Aynı Catalyst optimizer'a gidebilirler. Takım okunabilirliği ve reuse ihtiyacına göre seçim yap.

37. Missing values

```
clean = (
    df.na.fill({"country": "UNKNOWN"})
      .na.drop(subset=["customer_id"])
)
```

İşlem	PySpark
Fill	df.na.fill
Drop	df.na.drop
Replace	df.na.replace

38. Duplicate handling

```
dedup = df.dropDuplicates(
    ["event_id"]
)
```

Event semantics
Sadece duplicate key değil, hangi kaydın tutulacağı gerekiyorsa window + row_number kullan.

39. row_number ile dedup

```
w = (
    Window
    .partitionBy("event_id")
    .orderBy(F.col("ingested_at").desc())
)

dedup = (
    df.withColumn("rn", F.row_number().over(w))
      .filter(F.col("rn") == 1)
      .drop("rn")
)
```

40. Date/time işlemleri

```
df2 = (
    df.withColumn("date", F.to_date("ts"))
      .withColumn("hour", F.hour("ts"))
      .withColumn("week", F.weekofyear("ts"))
)
```

Timezone
Distributed event pipeline'larında timezone semantics baştan standartlaştırılmalıdır.

41. Büyük ölçekte feature engineering

Spark, milyonlarca kullanıcı için batch feature üretiminde güçlüdür.

```
features = (
    orders.groupBy("customer_id")
    .agg(
        F.countDistinct("order_id").alias("orders_90d"),
        F.sum("amount").alias("spend_90d"),
        F.avg("amount").alias("avg_basket")
    )
)
```

ML leakage
Prediction timestamp'e göre filtreleme ve historical point-in-time correctness yine zorunludur.

42. Spark MLlib

Spark'ın distributed machine learning kütüphanesidir.

Alan	Örnek
Feature	VectorAssembler, StringIndexer
Model	LogisticRegression, RandomForest
Clustering	KMeans
Pipeline	Transformer/Estimator pipeline

Modern pratik
Çoğu veri bilimci Spark'ı data prep için, model training'i XGBoost/LightGBM/PyTorch gibi ayrı stack'te kullanabilir. MLlib yine ölçekli baseline için değerlidir.

43. VectorAssembler

```
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(
    inputCols=["age","income","spend_90d"],
    outputCol="features"
)

ml_df = assembler.transform(df)
```

44. Spark ML pipeline

```
from pyspark.ml import Pipeline
from pyspark.ml.classification import LogisticRegression

lr = LogisticRegression(
    featuresCol="features",
    labelCol="label"
)

pipe = Pipeline(
    stages=[assembler, lr]
)

model = pipe.fit(train_df)
```

Pipeline farkı
scikit-learn mantığına benzer ama distributed DataFrame üzerinde çalışır.

45. Spark vs Pandas

Kriter	Pandas	Spark
Execution	Tek node	Distributed
Overhead	Düşük	Cluster/scheduler
API	Eager ağırlıklı	Lazy
Veri boyutu	RAM sınırı	Cluster ölçeği
Debug	Kolay	Daha karmaşık

Doğru araç
Spark büyük veri için; küçük/orta veri için Pandas/Polars genellikle daha hızlı geliştirme deneyimi sağlar.

46. Spark vs DuckDB/Polars

Modern single-node engines çok verimlidir. Distributed ihtiyacı yoksa Spark otomatik olarak en iyi seçenek değildir.

- Tek node RAM/disk sınırı içinde mi?
- Cluster mevcut mu?

- ETL süresi SLA'yı aşıyor mu?
- Parallel source scan faydalı mı?

Mühendislik kararı
Teknoloji seçimi veri boyutu kadar ekip operasyon maliyetiyle ilgilidir.

47. Memory management

- Executor memory
- Execution memory
- Storage/cache memory
- Off-heap/overhead

Belirti
OutOfMemoryError her zaman 'daha fazla RAM' demek değildir; skew, collect, cache ve partition boyutu da kontrol edilmeli.

48. Spill

Sort/aggregation için memory yetmezse ara data disk'e spill edilir.

Maliyet
Disk I/O artar. Spark UI'da spill metricleri performance debugging için değerlidir.

49. Spark UI

Sekme/metric	Ne anlarsın?
Jobs	Hangi action yavaş?
Stages	Shuffle/stage süreleri
Tasks	Skew, straggler
Storage	Cache kullanımı
Executors	Memory/GC/task dağılımı

Senior yaklaşım
Spark tuning'i config ezberiyle değil, Spark UI ve execution plan kanıtıyla yapılır.

50. Common configuration fikirleri

- spark.sql.shuffle.partitions
- executor cores/memory
- AQE
- broadcast threshold
- dynamic allocation

Uyarı
Her config'i elle değiştirmek performansı iyileştirmez. Önce bottleneck teşhis et.

51. Dynamic allocation

Workload'a göre executor sayısının artırılıp azaltılması yaklaşımıdır.

Trade-off
Kaynak verimliliği artar; startup/shuffle service ve platform detayları yönetilmelidir.

52. Checkpoint

Uzun lineage veya iterative computation'da dependency zincirini kesmek için checkpoint kullanılabilir.

Cache ile fark
Cache performance için; checkpoint lineage truncation ve fault recovery için daha kalıcı mantıktadır.

53. Structured Streaming

Spark'ın streaming API'si, sürekli gelen veriyi unbounded table gibi modellemeye çalışır.

Kavram	Anlam
Source	Kafka/files
Micro-batch	Küçük aralıklarla batch execution
Watermark	Late data sınırı
State	Window/aggregation state
Sink	Table/file/console

Batch bilgisi taşınır
DataFrame API'nin büyük kısmı streaming DataFrame'de de benzer mental modelle kullanılır.

54. Watermark

Late event'leri ne kadar bekleyeceğini belirleyerek streaming state'in sınırsız büyümesini engellemeye yardım eder.

Event time
Processing time yerine event time semantics gerektiren fraud/log pipeline'larında kritik.

55. Exactly-once sezgisi

Checkpoint, source offset ve idempotent/transactional sink davranışıyla duplicate-free sonuç hedeflenir.

Gerçek dünya
Exactly-once tek bir sihirli flag değildir; source-to-sink zincirinin semantics'ine bağlıdır.

56. Gerçek vaka: e-commerce daily sales

Adım	Spark kararı
Input	Partitioned Parquet orders
Filter	Tarih predicate pushdown
Join	Broadcast product dimension
Aggregate	store-product-day revenue
Write	Partition by sale_date
Quality	Row count + revenue sanity

Performans
Gereksiz columns join öncesi drop edilir; dimension küçükse broadcast kullanılır.

57. Gerçek vaka: customer 360

Orders, sessions, support ve profile tablolarından customer-level feature mart.

- Her source grain'i netleştir.
- Aggregate önce, sonra join.
- Hot customers için skew kontrol et.
- Point-in-time snapshots tut.

En yaygın hata
Raw one-to-many tabloları doğrudan birbirine join edip Cartesian-like row explosion yaratmak.

58. Gerçek vaka: fraud streaming

İhtiyaç	Spark Structured Streaming
Transactions	Kafka source
Feature	10-minute txn count
Time	Event-time window
Late data	Watermark
Sink	Online/serving layer veya alerts

59. PySpark coding standards

- F.col / explicit aliases kullan.
- selectExpr/SQL kullanıyorsan okunabilirlik koru.
- UDF'yi son çare yap.
- DataFrame isimlerinde grain belirt.
- Reusable transformation function yaz.

- Schema ve testleri version control'da tut.

Testable transform
function(input_df) -> output_df şeklindeki saf DataFrame transformation'lar unit test'i kolaylaştırır.

60. Unit test örneği

```
def build_daily_sales(df):
    return (
        df.groupBy("store_id", "date")
          .agg(F.sum("amount").alias("revenue"))
    )

# test:
# küçük local SparkSession + 3-5 satır fixture
```

Küçük data ile test
Dağıtık pipeline test etmek için dev cluster gerekmez; logical transformation küçük local fixture ile doğrulanabilir.

61. Performance checklist

1. Execution plan'da gereksiz scan var mı?
2. Filter mümkün olduğunca erken mi?
3. Gereksiz kolonlar düşürüldü mü?
4. Join strategy doğru mu?
5. Broadcast fırsatı var mı?
6. Skew var mı?
7. Shuffle volume büyük mü?
8. Partition sayısı uygun mu?
9. Small files var mı?
10. Cache gerçekten reuse edilen data için mi?
11. Python UDF yerine built-in var mı?
12. Spark UI'da spill/straggler var mı?

62. Sık yapılan hatalar

- Büyük DataFrame üzerinde collect() kullanmak.
- Her DataFrame'i cache etmek.
- Join öncesi gereksiz kolonları taşımak.
- Küçük dimension için broadcast fırsatını kaçırmak.
- groupByKey benzeri pahalı pattern'ler kullanmak.
- Skew'i yalnız partition sayısı artırarak çözmeye çalışmak.
- Python UDF'yi built-in function yerine kullanmak.
- Single-node probleme Spark dayatmak.
- Partition/file layout'i görmezden gelmek.

- Execution plan ve Spark UI bakmadan config tuning yapmak.

63. Mülakat soruları

Soru	Kısa cevap
Driver ne yapar?	Application coordination ve execution planning.
Executor nedir?	Worker process; task çalıştırır.
Transformation vs action?	Lazy plan oluşturma vs execution tetikleme.
Shuffle nedir?	Partition'lar arasında network data exchange.
Narrow vs wide?	Tek parent dependency vs multi-partition dependency.
repartition vs coalesce?	Genel shuffle repartition; azaltmada coalesce daha ucuz olabilir.
Broadcast join?	Küçük tabloyu executor'lara kopyalayıp büyük shuffle'ı önleme.
Skew nedir?	Key distribution dengesizliği nedeniyle straggler task.
Catalyst?	Spark SQL logical/physical plan optimizer.
AQE?	Runtime statistics ile adaptive optimization.
Cache ne zaman?	Aynı expensive DataFrame birden fazla action'da reuse edilirse.
collect neden riskli?	Tüm data driver memory'sine gelir.
Parquet avantajı?	Columnar scan, compression, pruning/pushdown.
Python UDF neden yavaş?	Serialization + optimizer sınırı.
Spark UI ne için?	Job/stage/task/memory/shuffle diagnosis.

64. Uygulama alıştırmaları

13. Local PySpark ile 1M satırlık synthetic dataset oluştur.
14. filter/select/groupBy pipeline kur ve explain planını incele.
15. repartition sayılarını değiştirip task dağılımını gözlemle.
16. Küçük dimension ile normal join ve broadcast join karşılaştır.
17. Skewed key dataset oluşturup task sürelerini karşılaştır.
18. Window ile customer previous transaction feature üret.
19. Parquet'e partitionBy(date) ile yaz ve pruning dene.
20. Built-in function ile Python UDF performansını kıyasla.
21. Cache kullanılan ve kullanılmayan tekrar action akışını ölç.
22. Spark UI üzerinden shuffle read/write metriclerini raporla.

65. Mini portföy projesi

Proje: Large-Scale E-Commerce Feature Pipeline with PySpark

Dosya	İçerik
src/read.py	Schema + partitioned Parquet reads
src/clean.py	Quality, dedup, casting
src/features.py	Customer/product aggregates
src/joins.py	Broadcast + skew-safe joins
src/write.py	Partitioned Gold outputs
tests/	Small Spark fixtures
notebooks/performance.ipynb	Explain + Spark UI findings
README.md	Architecture, bottlenecks, optimization decisions

CV cümlesi
Built and optimized a distributed PySpark feature pipeline using partitioned Parquet, broadcast joins, window aggregations, skew-aware transformations, execution-plan analysis and Spark UI performance diagnostics.

66. Final cheat sheet

Kavram	Hatırlanacak fikir
Driver	Plan + coordinate
Executor	Distributed worker
Partition	Parallel data unit
Transformation	Lazy operation
Action	Execution trigger
Catalyst	Query optimizer
Shuffle	Network redistribution
Narrow	No cross-partition exchange
Wide	Shuffle likely
Broadcast	Small-side join optimization
Skew	Uneven key distribution
AQE	Runtime adaptive plan
Cache	Reuse expensive result
Parquet	Columnar distributed storage
Pushdown	Filter scan reduction
Pruning	Read only needed columns/partitions
Spark UI	Evidence-based tuning
Structured Streaming	Streaming DataFrame model

Bu modülden sonra
36. Cloud bölümünde Spark workload'larının AWS/Azure/GCP üzerinde storage, compute ve managed data platformlarıyla nasıl konumlandığını öğreneceksin.